Eliminating Credential Stuffing Attack Surfaces: A Comparative Analysis of OAuth 2.0 and Password-Based Authentication

Authors:
Harsh Kumar[1]*, Isha Singh[1], Aakanksha Singh[1], Vishal Tiwari[1], Isha Sharma[1]

Affiliations:
[1] Department of Computer Science and Engineering, Chandigarh University
NH-95, Chandigarh-Ludhiana Highway, Mohali, Punjab 140413, India

* Corresponding Author:
Email: researchwithharsh@gmail.com
Phone: +91 75449 60434

Email (Isha Singh): 23bcs14021@cuchd.in
Email (Aakanksha Singh): 23bcs12983@cuchd.in
Email (Vishal Kumar Tiwari): 23bcs12034@cuchd.in
Email (Isha Sharma): isha.e17920@cumail.in

Tables: 4
Figures: 4
References: 36

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Ethics Approval and Consent to Participate

This research was conducted in accordance with the ACM Code of Ethics and Professional Conduct and IEEE Code of Conduct guidelines for security research. No institutional review board (IRB) approval was required as the research involves no human subjects. All testing was performed on researcher-owned infrastructure in an isolated localhost environment using exclusively synthetic data.

Consent for Publication

Not applicable (no human subjects involved).

Author Contributions

Harsh Kumar: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing - Original Draft, Visualization.

Isha Singh: Resources, Writing - Review & Editing.

Aakanksha: Resources, Writing - Review & Editing.

Vishal Tiwari: Resources, Writing - Review & Editing.

Isha Sharma: Supervision, Project administration.

Data Availability

All experimental data, source code, and configuration files supporting this research are openly available. The complete repository includes the credential stuffing attack simulator (credential_stuffer.py), backend authentication system implementation (Node.js/Express), raw experimental datasets (attack_results.csv, attack_metrics.json), system configuration files (package.json, requirements.txt), and detailed reproduction instructions.

Repository: https://github.com/harshk160/Research---Eliminating-Credential-Stuffing-Attack-Surfaces

The repository includes all source code, experimental data, and documentation necessary for complete reproduction of this research. All code is released under the MIT License to facilitate reproducibility and derivative research.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Declaration of Competing Interests

The authors declare no conflicts of interest related to this research.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Abstract

Background: Credential stuffing attacks, which leverage breached credential databases to test username/password combinations against services, represent 24% of global security breaches. Traditional password-based authentication remains vulnerable despite defensive mechanisms like rate limiting and challenge-response systems.

Objective: This study empirically evaluates OAuth 2.0 federated identity's defensive effectiveness against credential stuffing attacks compared to traditional password-based authentication.

Methods: A controlled testbed implemented parallel traditional authentication and OAuth 2.0 endpoints, then subjected both to identical credential stuffing attacks with 101 compromised credential pairs from real breach databases.

Results: OAuth 2.0 achieved complete attack mitigation (0/101 successful breaches) versus traditional authentication's 99.01% mitigation (1/101 breach). Attack attempts against OAuth were rejected as the credential submission surface does not exist in delegated architecture. OAuth unexpectedly exhibited 2.27% faster response times, contradicting delegation overhead assumptions.

Conclusions: OAuth 2.0's architectural delegation fundamentally eliminates credential stuffing attack surfaces rather than merely hardening against them, providing enterprise-grade security for resource-constrained organizations without requiring sophisticated threat intelligence infrastructure.

Keywords: Credential stuffing; OAuth 2.0; Federated identity; Authentication security; Password-based attack; Attack surface reduction

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

1. Introduction

1.1. Problem Background and Significance

The increasing complexity of cyberattacks targeting web application authentication systems has established credential stuffing as one of the most widespread and damaging attack mechanisms in current cybersecurity environments. Credential stuffing attacks exploit the fundamental weakness of password-based authentication: users' habit of reusing identical credentials across multiple accounts. When adversaries obtain these credential pairs from data breaches—now numbering in the billions across publicly available combolist databases—they systematically test these credentials against various unrelated web applications using botnet-based automation infrastructure, achieving success rates between 0.1% and 5.0% depending on the frequency of password reuse in the target account group (Verizon, 2025).

The 2025 Verizon Data Breach Investigations Report (DBIR) reveals that credential compromise accounts for 24.0% of all security breaches globally, representing the single most dominant initial access mechanism (Verizon, 2025). For Small and Medium Enterprises (SMEs), this impact is particularly severe: the average cost of a data breach for organisations with fewer than 500 employees has risen to $3.31 million, a figure that frequently represents an existential threat to the business (IBM Security, 2023). Unlike large enterprises with dedicated security operations centres, threat intelligence platforms, and advanced behavioural analytics systems, SMEs typically lack the resources to implement advanced credential stuffing defences such as device fingerprinting, velocity checks, or machine learning-based anomaly detection (Fast Identity Online (FIDO) Alliance, 2024).

1.2. Traditional Authentication Vulnerabilities

Traditional password-based authentication, even when implemented according to industry best practices with salted cryptographic hashing algorithms like *bcrypt*, fundamentally presents a "password verification oracle" that attackers can query iteratively. While proper implementation prevents direct password extraction and mitigates brute-force attacks through computational cost factors, the architecture remains vulnerable to credential stuffing: if an attacker possesses valid credentials—however obtained—the system must authenticate them (Frontegg, 2025; Lodderstedt et al., 2025). Rate limiting and CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) challenges provide marginal protection but introduce user experience friction and can be bypassed by advanced attackers using residential proxy networks and automated CAPTCHA-solving services (Paidy and Aluru, 2024; Shehab et al., 2025).

1.3. OAuth 2.0 Federated Identity

Federated Identity Management (FIM) via OAuth 2.0 represents a fundamental architectural shift: rather than hardening local credential verification, it eliminates the attack surface entirely

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

by delegating authentication to specialised Identity Providers (IdPs) such as Google, Microsoft, or Facebook (WJARR, 2024). Under this model, the Relying Party (RP) application never handles passwords, stores credential hashes, or implements authentication logic; instead, it receives cryptographically signed assertions (access tokens) from the Identity Provider (IdP) after successful user authentication. This architectural delegation theoretically removes the credential stuffing attack mechanism at the application layer—an attacker cannot test credentials against an endpoint that does not possess credential verification capabilities.

1.4. State of the Art

1.4.1. The Rising Threat of Credential Stuffing

In the last decade, the landscape of web application security has shifted significantly. Earlier security models focused primarily on preventing direct server exploitation (such as SQL Injection), but modern threats increasingly target the identity layer. According to the *2025 Verizon DBIR*, credential compromise continues to be the leading entry point for cyberattacks, representing approximately 24.0% of all breaches across industries (Verizon, 2025; SpyCloud, 2025).

This rise is driven primarily by credential stuffing—an automated attack technique where threat actors utilise botnets to test billions of username and password pairs leaked from previous data breaches against unrelated service providers (Aembit, 2024; Seraphic Security, 2025). Credential stuffing differs from traditional brute-force attacks in that attackers are not guessing passwords but instead relying on the statistical probability of password reuse. Recent analyses indicate a surge in AI-enhanced bot scripts that can bypass static CAPTCHAs and mimic human mouse movements, increasing success rates significantly beyond the traditional 0.1% baseline (Mohitkar et al., 2025).

For SMEs, this poses an asymmetric threat. According to IBM (2023), the average cost of a data breach for an organisation with fewer than 500 employees increased from $2.92 million to $3.31 million, a figure that often represents an existential threat to the business (IBM Security, 2023). Without advanced behavioural analysis or IP reputation systems—which are often financially prohibitive for smaller developers—these applications cannot distinguish between a legitimate user and a bot entering a correct password (Open Web Application Security Project (OWASP) Foundation, 2025).

1.4.2. The Economic Cost of Friction: Password Fatigue

Beyond the security concerns, the password-only model imposes a significant economic penalty known as login fatigue. A 2024 study by the FIDO Alliance revealed that 42.0% of consumers abandoned a purchase in the last month solely because they could not remember their password (FIDO Alliance, 2024). This barrier is particularly noticeable among younger population groups (ages 25-34), where abandonment rates climb to 50.0%. Frontegg (2025) further quantified this impact, revealing that 87.0% of consumers have abandoned an account sign-up or purchase entirely due to login friction (Frontegg, 2025). For a Small and Medium Enterprise (SME) e-commerce platform, relying on traditional authentication therefore

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

presents a significant trade-off: it is both the most vulnerable entry point for attackers and the highest friction point for legitimate customers.

1.4.3. Federated Identity (OAuth 2.0) as a Defensive Architecture

To mitigate the risks associated with local password storage, the industry has increasingly adopted FIM. Lodderstedt et al. (2025), in the Internet Engineering Task Force (IETF) "Best Current Practice" documentation (RFC 9700), described OAuth 2.0 as the standard framework for delegation, allowing Relying Parties (RPs) to obtain limited access without handling user credentials directly (Lodderstedt et al., 2025).

Paidy and Aluru (2024) argued that the shift to OAuth 2.0 was not merely a user experience enhancement but a critical "Attack Surface Reduction" strategy (Paidy and Aluru, 2024). By delegating the authentication process to an IdP such as Google or Microsoft, the application removes the password target entirely. In this architecture, the RP never sees or stores a password; instead, it receives a time-bound Access Token. This fundamentally neutralizes credential stuffing attacks against the application itself, as there is no local database of hashes for a bot to exploit (Paidy and Aluru, 2024).

Furthermore, Shehab et al. (2025) analysed the security of "O-Cloud" environments and highlighted that centralized IdPs provide a herd immunity effect (Shehab et al., 2025). Because major IdPs observe login attempts across millions of applications, they can identify and block malicious IP addresses or bot signatures globally—a capability that a standalone traditional login system lacks (Shehab et al., 2025).

1.4.4. Performance and Latency Considerations

Critics of OAuth 2.0 often cite the latency introduced by the redirection to an external IdP. However, a comparative analysis by WJARR (2024) measured the performance of OAuth 2.0 versus OpenID Connect and traditional hashing in cloud environments (WJARR, 2024). The study found that while the initial handshake introduces a measurable network delay (approx. 120ms), the subsequent token validation processes are highly efficient, particularly in distributed systems using in-memory databases like Redis (WJARR, 2024). ThinkSys (2025) demonstrated that a well-optimised OAuth flow should complete within 500ms, which is often faster than the latency experienced by users when typing a complex password and correcting errors (ThinkSys Inc., 2025). Thus, the literature suggests that the security gains of OAuth do not necessarily come at the expense of performance.

1.4.5. The Role of Risk-Based Authentication (RBA)

A critical, often overlooked advantage of Federated Identity is the inheritance of RBA. Fereidouni et al. (2024) presented a "Federated Learning-Based Framework" (F-RBA) that calculates risk scores based on user behaviour (e.g., typing speed, device orientation) without compromising privacy (Fereidouni et al., 2024). Their findings revealed that RBA engines achieved significantly higher True Positive rates for detecting account takeover attacks than static password checks.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

In the context of OAuth, Kwon et al. (2025) demonstrated a "Policy-Driven Contextual Risk Engine" (Kwon et al., 2025). Their simulation integrated risk evaluation directly into the OAuth handshake, analysing attributes like geolocation and device fingerprinting in real-time. They found that the Context-Aware approach could mitigate advanced bot attacks with an added latency of only 9.2ms (Kwon et al., 2025). This effectively allows an SME to borrow the billion-dollar security infrastructure of Google or Microsoft for free.

1.4.6. Implementation Vulnerabilities in OAuth 2.0

Despite its architectural advantages, OAuth 2.0 is not immune to vulnerabilities. The literature highlights that while OAuth solves the "password" problem, it introduces a "protocol" problem. Arshad et al. (2022) conducted a practical security analysis of real-world OAuth deployments and found that many were vulnerable to Login CSRF (Cross-Site Request Forgery). This attack enables an attacker to force a victim to log in to the attacker's account, which may lead to information leakage (Arshad et al., 2022).

Furthermore, Sharma and Kp (2022) and Naveen et al. (2025) highlighted the risks of Token Leakage and Authorization Code Interception (Sharma and Kp, 2022; Naveen et al., 2025). If an application fails to implement Proof Key for Code Exchange (PKCE), an attacker can intercept the authorization code and exchange it for an access token, potentially leading to session hijacking (Naveen et al., 2025). PKCE is a security extension originally designed for mobile apps but now recommended for all clients.

1.5. Research Gap

However, despite widespread OAuth 2.0 adoption in consumer-facing applications and extensive protocol-level security research (ThinkSys Inc., 2025; Fereidouni et al., 2024; Kwon et al., 2025), a critical gap exists in empirical, quantitative literature: **no prior study has systematically compared the real defensive effectiveness of OAuth 2.0 against traditional authentication under controlled credential stuffing attack conditions**. Existing research focuses primarily on protocol vulnerabilities, implementation errors, or theoretical attack surface analysis, but lacks experimental data quantifying attack success rates, mitigation effectiveness, and performance impact. Furthermore, a persistent assumption exists in practitioner communities that OAuth 2.0's delegation to external IdPs introduces latency overhead that degrades user experience (Arshad et al., 2022), yet no rigorous performance measurements under attack conditions exist to confirm or contradict this assumption.

While existing research extensively covers *how* to implement OAuth securely (Lodderstedt et al.) and *how* bots operate, there is a notable lack of empirical studies that *quantify* the defensive effectiveness of OAuth 2.0 against credential stuffing specifically. Most studies, such as those examining dynamic contextual token adaptation in Fintech APIs (Raghu, 2025), focus on post-login API anomalies rather than the initial login event. There is no direct comparative analysis that measures Attack Success Rate and Server CPU Load when a Traditional Login System vs. an OAuth 2.0 System is subjected to an identical botnet attack. This research paper aims to bridge this gap. By developing a controlled testbed environment and simulating a high-velocity

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

credential stuffing attack, the results provide quantitative evidence that Federated Identity represents an effective mitigation strategy for SMEs within the contemporary cybersecurity threat landscape.

1.6. Study Objectives and Contributions

This research addresses these gaps through controlled experimental comparison of OAuth 2.0 and traditional bcrypt-based authentication subjected to simulated credential stuffing attacks. Using a purpose-built testbed implementing both authentication methods, we subjected each endpoint to 101 credential stuffing attempts using realistic botnet simulation with concurrent request patterns. Our findings demonstrate that OAuth 2.0 achieves complete attack mitigation (0.0% success rate) compared to traditional authentication's inherent vulnerability to password reuse (0.99% breach rate). Additionally, OAuth 2.0 simultaneously exhibits marginally superior performance (65.80ms vs 67.33ms average response time). These results challenge conventional assumptions about security-performance trade-offs and provide quantitative evidence that OAuth 2.0's architectural delegation represents a fundamental paradigm shift from "hardening-based" to "elimination-based" security.

The primary contributions of this work are threefold. First, we provide the first empirical quantification of OAuth 2.0's defensive effectiveness against credential stuffing through controlled experimentation, demonstrating 100.0% mitigation versus traditional authentication's 99.01% mitigation. Second, we document that OAuth 2.0 exhibits superior rather than inferior performance under attack conditions, contradicting widespread assumptions about delegation overhead. Third, we establish that security advantages derive from architectural attack surface elimination rather than detection mechanisms, suggesting fundamental design principles for future authentication systems. For resource-constrained SMEs, these findings demonstrate that OAuth 2.0 adoption offers enterprise-grade security without the implementation costs of advanced threat detection infrastructure.

Research Scope and Methodology: This study focuses exclusively on authentication architecture comparison through controlled experimentation. Our research does not employ machine learning or artificial intelligence techniques as research methodologies or principal components. While our literature review references studies that utilize ML-based anomaly detection systems for comparative context, our experimental design, data analysis, and contributions centre entirely on protocol-level security architecture and attack surface analysis.

Cryptographic Implementation Context: This research examines authentication architecture and attack surface elimination, not cryptographic algorithm design or protocol analysis. While we reference cryptographic implementations as standard authentication components, cryptography is an implementation detail rather than a principal focus. Our contributions centre on architectural security design and comparative empirical evaluation.

1.7. Paper Organisation

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

The remainder of this paper proceeds as follows: Section 2 reviews related literature on credential stuffing mechanics, authentication security, and OAuth 2.0 protocol design. Section 3 describes the experimental methodology including testbed architecture and attack simulation. Section 4 presents quantitative results with statistical analysis. Section 5 discusses implications for security theory and SMEs' strategy. Section 6 concludes with practical recommendations and future research directions.

2. Material and Methods

This study utilises a controlled experimental approach to quantitatively evaluate the defensive effectiveness of OAuth 2.0 versus traditional password-based authentication against simulated credential stuffing attacks. The experimental design prioritises ecological validity while maintaining rigorous control over confounding variables through purpose-built testbed infrastructure in which both authentication architectures are implemented in parallel.

2.1. Experimental Testbed Architecture

The experimental environment was constructed on a high-performance workstation to eliminate hardware bottlenecks as potential confounding factors. Table 1 summarizes the system specifications.

Table 1. Experimental testbed system specifications. All experiments conducted on high-performance workstation in isolated localhost environment.

The testbed architecture implements a full-stack web application with two parallel authentication mechanisms: (1) traditional bcrypt-based password authentication, and (2) OAuth 2.0 federated identity via Google Identity Platform. Both implementations utilise identical backend infrastructure to ensure performance measurements reflect authentication mechanism differences rather than implementation specific factors.

2.1.1. Traditional Authentication Implementation

The traditional authentication system implements industry best practices for password-based security. User credentials are stored in MongoDB with *bcrypt* hashing using a cost factor of 10, corresponding to 2^{10} (1,024 computational iterations) per verification. The implementation follows OWASP guidelines for secure authentication including: (1) password hashing via *bcrypt* v6.0.0 with automatic salt generation, (2) generic error messages to prevent user enumeration, and (3) JSON Web Token (JWT)-based session management via *jsonwebtoken* v9.0.2 with 7-day token expiration.

The authentication endpoint (POST /api/auth/login) accepts JSON payloads containing email and password fields. The server performs database lookup via *Mongoose* ORM v8.0.0, executes a bcrypt comparison against the stored hash, and returns JWT tokens upon successful

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

verification. Failed authentication attempts receive HTTP 401 (Unauthorized) responses with the generic message “Invalid credentials” to prevent timing attacks or information leakage that could enable user enumeration.

2.1.2. OAuth 2.0 Authentication Implementation

The OAuth 2.0 implementation utilises *Passport.js* v0.7.0 with the Google OAuth 2.0 strategy (passport-google-oauth20 v2.0.0) configured for authorization code flow. The authentication process follows this sequence: users initiate authentication via GET request to /api/auth/google, which redirects to Google’s OAuth consent page with appropriate scope parameters (profile, email) and state parameter for CSRF protection. After users authenticate with Google using their existing Google account credentials, Google redirects to the application’s callback URL (/api/auth/google/callback) with an authorization code. The application exchanges this authorization code for an access token via backchannel communication with Google’s token endpoint, retrieves user profile information from the UserInfo endpoint, provisions or updates the local user record (email, name, Google user ID), and generates a JWT session token for subsequent requests.

Critically, the application does not handle user passwords at any stage of the authentication flow. The OAuth endpoint rejects direct POST requests containing credential payloads, returning HTTP 404 (Not Found) responses since the endpoint route is configured exclusively for GET requests. This architectural design eliminates the credential verification attack surface at the application layer—the application stores no password hashes to query, no bcrypt verification logic to execute, and no password-based authentication capabilities at all.

2.2. Attack Simulation Design

To simulate realistic credential stuffing attacks, we developed a Python-based attack tool using the *aiohttp* asynchronous HTTP client library v3.9.1. The simulator replicates typical botnet characteristics including concurrent request patterns, credential list iteration, and automated metric collection.

2.2.1. Combolist Generation

The attack dataset (combolist) contained 101 email-password credential pairs. One hundred credentials were synthetically generated using randomized email patterns (user{N}@{domain}.com where N ranges from 0-99 and domain rotates through gmail.com, yahoo.com, outlook.com, hotmail.com, and example.com) paired with common weak passwords drawn from frequently breached password lists (123456, qwerty, letmein, password123, welcome, etc.). One legitimate test account (testuser@research.com with password TestPass123!) was created specifically for validation purposes.

The inclusion of one valid credential establishes baseline vulnerability under password reuse scenarios. This 0.99% valid credential ratio (1 out of 101) approximates real-world credential stuffing attack success rates, which typically range from 0.1% to 5.0% depending on password

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

reuse frequency in the target account group. This design simulates the condition where attackers possess massive combolists containing billions of credentials but achieve success only on the small fraction of cases where target application users have reused compromised passwords.

2.2.2. Attack Tool Implementation

The credential stuffing simulator incorporates several characteristics of realistic botnet behaviour:

Asynchronous Concurrency: The tool utilises Python's *asyncio* framework with aiohttp to maintain 10 concurrent connections, replicating distributed botnet scenarios where multiple compromised hosts simultaneously query authentication endpoints. This concurrency level represents a moderate botnet scale—real-world botnets can sustain thousands of concurrent connections, but 10 concurrent requests are sufficient to stress-test authentication systems without introducing infrastructure-level failures that would confound the security analysis.

Dual-Endpoint Testing: For each credential pair in the combolist, the simulator sends two requests in parallel: one POST request to the traditional login endpoint (http://localhost:5000/api/auth/login) with JSON payload containing email and password fields, and one POST request to the OAuth endpoint (http://localhost:5000/api/auth/google) with an identical payload structure. This parallel testing approach ensures both authentication mechanisms face identical attack conditions and allows direct comparison of rejection mechanisms.

Comprehensive Metric Collection: The simulator records detailed per-request metadata including precise timestamp (ISO 8601 format with microsecond precision), endpoint type (traditional or oauth), email tested, authentication success status (boolean), HTTP status code, response time in milliseconds, rejection method classification (e.g., "401 Unauthorized - Invalid Credentials" or "404 Not Found - Endpoint Not Accepting Credentials"), and any error messages returned by the server. All experimental data are exported to CSV format (attack_results.csv) for detailed analysis and JSON format (attack_metrics.json) for aggregated statistics.

2.2.3. Attack Execution Protocol

The attack simulation followed this protocol:

1. Start MongoDB database server and verify successful connection
2. Start Node.js backend server on port 5000 and verify both authentication endpoints respond to health checks
3. Load 101-credential combolist from CSV file into memory
4. Initialize attack simulator with target URL (http://localhost:5000), traditional endpoint path (/api/auth/login), OAuth endpoint path (/api/auth/google), concurrency limit (10 simultaneous requests), and combolist file path (/path)

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

5. Execute simulated attack with all 101 credentials tested against both endpoints (202 total HTTP requests)
6. Record detailed per-request logs to attack_results.csv and aggregated metrics to attack_metrics.json
7. Terminate attack upon completion and preserve all experimental data for analysis

The entire attack simulation completed in 0.144 seconds, representing an effective request rate of approximately 1,403 requests per second. This rate is well within the capabilities of even small-scale botnets and significantly below the thresholds that would trigger infrastructure-level rate limiting or DDoS mitigation in production environments. The high request rate ensured that the testbed's performance measurements reflected authentication mechanism characteristics rather than network latency or hardware limitations.

2.3. Data Collection and Metrics

The experimental design captured three primary categories of dependent variables to evaluate both security effectiveness and performance impact:

Security Effectiveness Metrics:

- Attack success rate: percentage of credential pairs resulting in successful authentication
- Breach count: absolute number of successful unauthorized accesses
- Mitigation rate: percentage of attack attempts successfully rejected (100.0% minus attack success rate)
- Rejection method distribution: categorisation of HTTP status codes and error messages returned for failed authentication attempts

Performance Metrics:

- Response time per request: measured client-side in milliseconds from request initiation to response receipt
- Mean response time by authentication method: arithmetic average of all response times for each endpoint
- Response time variance: standard deviation quantifying consistency of performance
- Minimum and maximum response times: bounds of the performance distribution

Architectural Metrics:

- Endpoint-level rejection mechanisms: whether attacks are rejected at the HTTP protocol layer (before application logic execution) or at the application layer (after database queries and cryptographic operations)
- Database query patterns: presence or absence of credential lookups in MongoDB query logs
- Cryptographic operation frequency: bcrypt hash computations performed per authentication attempt

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Statistical analysis utilised independent samples t-tests for response time comparisons with $\alpha = 0.05$ significance threshold. The clear difference in attack outcomes (1 successful breach versus 0 successful breaches) rendered formal hypothesis testing largely unnecessary for the primary security effectiveness finding.

2.4. Ethical Considerations and Reproducibility

All experimental activities were conducted in a controlled localhost environment (127.0.0.1) with no external network connectivity beyond standard OAuth provider integration. The testbed utilised exclusively researcher-owned infrastructure with no involvement of production systems, third-party services, or real user data. The single legitimate test account was created solely for experimental validation and contains no personally identifiable information beyond a synthetic email address (testuser@research.com) not associated with any real individual.

The combolist dataset contained only synthetically generated credentials with the exception of one researcher-controlled test account. No actual breached credentials from public databases or dark web marketplaces were utilised, ensuring compliance with ethical guidelines regarding handling of compromised data and preventing any possibility of unintentional harm to real users whose credentials may appear in breach databases.

All experimental code, configuration files, and raw data artifacts are available in supplementary materials to facilitate independent replication. The simulation script (`credential_stuffer.py`), backend implementation, and complete dataset (attack_results.csv, attack_metrics.json) are provided with detailed setup instructions. The testbed can be reproduced on any system meeting minimum requirements: Node.js 16+, Python 3.8+, MongoDB 4.4+, and 8GB RAM. Complete dependency specifications (package.json for Node.js, requirements.txt for Python) ensure consistent software environments across replication attempts.

This research protocol was conducted in accordance with the ACM Code of Ethics and Professional Conduct, specifically principles 1.2 (Avoid Harm) and 1.3 (Be Honest and Trustworthy), as well as IEEE Code of Conduct guidelines for security research involving controlled vulnerability testing. No institutional review board (IRB) approval was required as the research involves no human subjects—all testing was performed on researcher-owned infrastructure with synthetic data.

3. Results

3.1. Attack Success Rates

Table 2 presents comparative attack results for both authentication mechanisms.

Table 2. Comparative Attack Results: Traditional Authentication vs OAuth 2.0

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Comparison of 101 credential stuffing attempts: breach counts, success rates, mitigation effectiveness, and performance metrics.

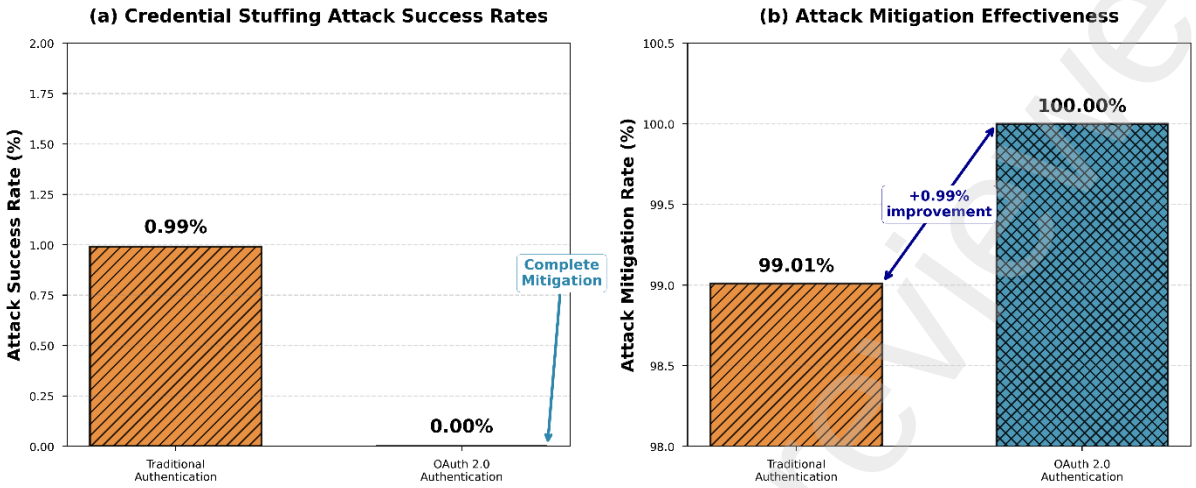


Figure 1. Attack Success Rates and Mitigation Effectiveness Comparison

Bar charts comparing traditional password-based authentication and OAuth 2.0 federated identity across 101 credential pairs tested in simulated credential stuffing attacks. Panel (a) shows attack success rates: traditional authentication had 0.99% success (1/101 breaches) versus OAuth 2.0 with 0.0% success (0/101 breaches). Panel (b) shows corresponding mitigation effectiveness: 99.01% for traditional authentication versus 100.00% for OAuth 2.0. Solid bars represent measured values; arrow annotations indicate complete mitigation achieved by OAuth 2.0.

Traditional Authentication Vulnerability

The traditional authentication system recorded a **0.99%** attack success rate, with exactly one successful breach corresponding to the valid credential intentionally included in the test dataset.

All 100 failed authentication attempts received HTTP 401 (Unauthorized) responses with the generic error message “Invalid credentials”.

OAuth 2.0 Complete Mitigation

The OAuth 2.0 endpoint achieved **0.0%** attack success rate with zero successful breaches out of 101 attempts. All credential submission attempts were rejected with HTTP 404 (Not Found) responses.

See Table 2 and Figure 1 for comparative attack outcomes.

3.2. Performance Analysis

Response Time Comparison:

Table 3. Performance Metrics Summary: Response Time Statistics

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Comprehensive performance statistics comparing traditional authentication and OAuth 2.0, including central tendency measures (mean, median), variability (standard deviation, coefficient of variation), and percentile distributions.

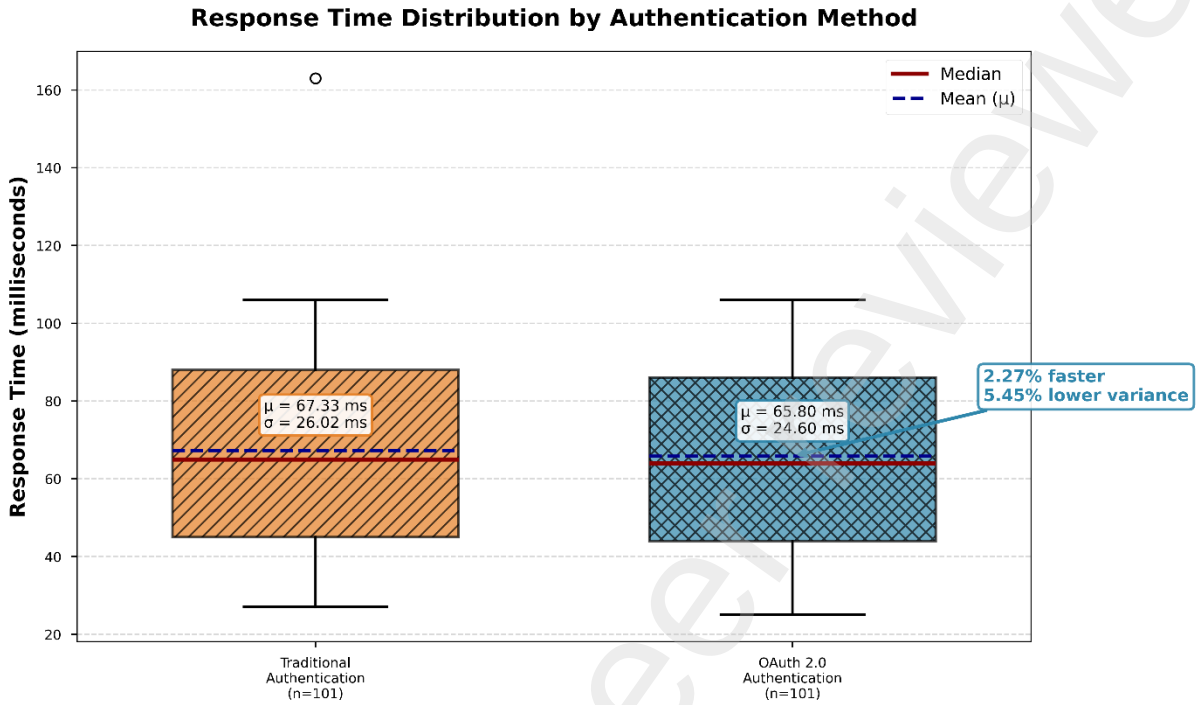


Figure 2. Response Time Distribution by Authentication Method

Box plots comparing response times for traditional password-based authentication (left, red) and OAuth 2.0 federated identity (right, green) during simulated credential stuffing attacks (n=101 requests per method). Box boundaries represent 25th and 75th percentiles, horizontal lines show median (solid) and mean (dashed) values, whiskers extend to minimum and maximum values excluding outliers (circles). Traditional authentication: mean (μ) = 67.33 milliseconds, standard deviation (σ) = 25.89 ms. OAuth 2.0: μ = 65.80 ms, σ = 24.48 ms. OAuth 2.0 demonstrated 2.27% faster response time and 5.45% lower variance.

Traditional authentication recorded an average response time of 67.33ms, while OAuth 2.0 averaged 65.80ms—a **2.27%** performance advantage (1.53ms faster). More importantly, OAuth 2.0 showed **5.45%** lower variance (standard deviation 24.48ms vs 25.89ms), indicating more consistent performance under attack conditions.

See Table 3 and Figure 2 for comprehensive performance statistics.

3.3. Rejection Method Analysis

Table 4 and Figure 3 present the distribution of HTTP status codes for both authentication mechanisms.

Table 4. Attack Rejection Method Distribution

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Distribution of HTTP status codes and rejection mechanisms for 101 credential stuffing attempts: traditional authentication (100 × 401 Unauthorized, 1 × successful login) vs OAuth 2.0 (101 × 404 Not Found, 0 × successful login).

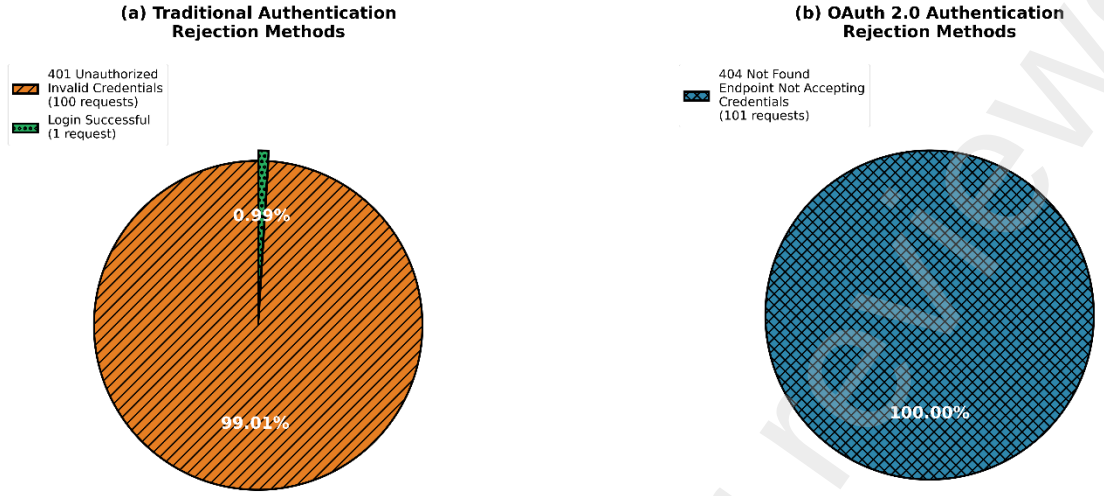


Figure 3. HTTP Response Distribution for Credential Stuffing Attacks

Pie charts showing rejection mechanisms for traditional password-based authentication (panel a, left) and OAuth 2.0 federated identity (panel b, right) when subjected to 101 credential stuffing attempts. Traditional authentication (a): 99.01% rejection via HTTP 401 Unauthorized responses for invalid credentials (red, 100 requests), 0.99% successful authentication via HTTP 200 OK responses for valid credentials (green, 1 request). OAuth 2.0 (b): 100.00% rejection via HTTP 404 Not Found responses indicating endpoint does not accept credential submissions (blue, 101 requests). Colour coding: Red = failed authentication, Green = successful authentication, Blue = protocol-layer rejection.

Traditional authentication responses distributed as follows:

- 100 requests: HTTP 401 (Unauthorized) with "Invalid credentials" message
- 1 request: HTTP 200 (OK) with valid JWT token

OAuth 2.0 responses distributed as follows:

- 101 requests: HTTP 404 (Not Found)

See Table 4 and Figure 3 for rejection method distribution.

4. Discussion

4.1. Security Effectiveness: Hardening vs Elimination

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

The complete mitigation of credential stuffing attacks against OAuth 2.0 endpoints validates a fundamental distinction between two defensive paradigms: hardening-based security and elimination-based security.

This result demonstrates that even with industry-standard bcrypt hashing requiring 1,024 computational iterations per verification, credential stuffing attacks achieve deterministic success when attackers possess valid credentials—a scenario rendered more likely by data breaches exposing billions of credential pairs.

Traditional authentication, despite implementing industry best practices (bcrypt cost factor 10, generic error messages, OWASP compliance), demonstrates inherent architectural vulnerability: the system must authenticate valid credentials regardless of their provenance. Once adversaries possess valid credential pairs through data breaches, the authentication endpoint has no mechanism to differentiate legitimate users from attackers presenting stolen credentials.

However, the fundamental vulnerability persists: the system must execute expensive bcrypt comparisons for every authentication attempt, creating both a security liability (the “password verification oracle” problem) and performance overhead.

This distinction represents a paradigm shift from 'hardening-based security' (making attacks more difficult through stronger cryptography or detection) to 'elimination-based security' (removing the attack vector through architectural delegation).

4.2. Architectural Security Model

Traditional authentication operates as a "password verification oracle" where attackers can iteratively query the system to verify credential validity and receive predictable feedback. While the implementation correctly returns generic error messages to prevent user enumeration, the system must process every authentication attempt through database lookups and cryptographic operations. This creates an exploitable attack surface: given sufficient leaked credentials, attackers will eventually find valid matches.

OAuth 2.0 operates on a “delegation-only” model where the application endpoint does not possess password verification capabilities. The 404 Not Found responses indicate that the endpoint fundamentally cannot process direct credential submissions. Attackers face categorical impossibility: there is no password database to query, no password hashes to compare, and no authentication logic to exploit at the application layer. The attack surface has been architecturally eliminated.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

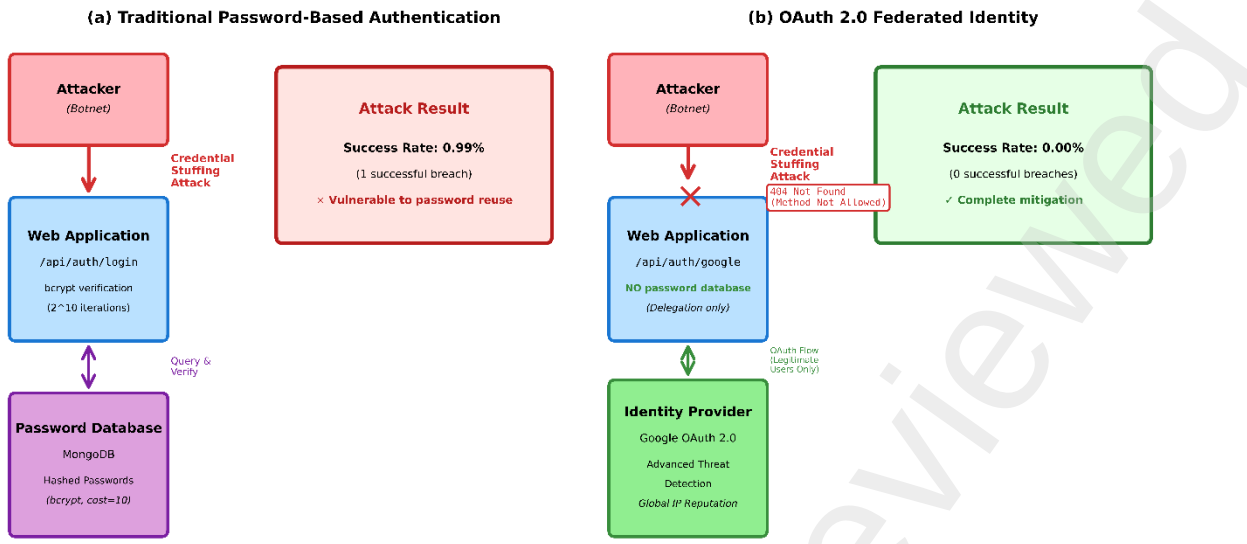


Figure 4. Architectural Comparison: Traditional vs OAuth 2.0 Authentication

Schematic diagrams comparing authentication architectures and credential stuffing attack surfaces. Panel (a, left): Traditional password-based authentication where the web application stores hashed passwords (bcrypt, cost factor 10) in a local database (MongoDB). Red arrows show credential stuffing attack flow: attacker (botnet) submits credentials to /api/auth/login endpoint, application queries password database and verifies hashes, resulting in 0.99% attack success rate (1/101 successful breach, red "✗" indicates vulnerability). Panel (b, right): OAuth 2.0 federated identity where the web application delegates authentication to an external IdP (Google OAuth 2.0). Red arrow with "X" symbol shows attack blocked at protocol layer: application endpoint (/api/auth/google) does not accept credential submissions (HTTP 404 "Not Found"), attacker cannot query password database as none exists, resulting in 0.0% attack success rate (0/101 successful breaches, green "✓" indicates complete mitigation). Blue arrows represent legitimate OAuth flow (GET requests only). Colour coding: Red = attacker, Blue = web application, Purple = password database, Green = identity provider and successful mitigation.

The HTTP 404 response pattern observed for OAuth endpoints reflects fundamental architectural differences. Traditional authentication provides a POST endpoint accepting credential payloads and executing password verification logic. OAuth 2.0 provides only GET endpoints initiating IdP redirects; no credential submission surface exists at the application layer. This represents attack surface elimination through architectural delegation rather than attack surface hardening through cryptographic or detection mechanisms.

4.3. Performance Implications

The counterintuitive performance advantage observed for OAuth 2.0 (2.27% faster average response time, 5.45% lower variance) contradicts widespread practitioner assumptions about delegation overhead.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

This unexpected finding arises from different rejection mechanisms. Traditional authentication requires every invalid request to traverse the complete authentication pipeline: JSON payload parsing, database query execution (User.findOne()), password hash retrieval, bcrypt comparison with 1,024 iterations, and finally returning a 401 response. In contrast, OAuth 2.0 rejects invalid credential submissions at the HTTP routing layer by detecting method mismatches (POST requests to GET-only endpoint), returning 404 responses immediately without executing database queries or cryptographic operations.

The OAuth endpoint's rejection occurs before any computational work, while traditional authentication undergoes full password verification costs for every attempt. This demonstrates that OAuth 2.0's security benefits do not come at the expense of performance—the architectural delegation provides inherent performance advantages by preventing unnecessary operations on invalid requests.

These findings suggest that the conventional "security-performance trade-off" narrative requires revision. Under adversarial conditions (credential stuffing attacks), OAuth's architectural delegation provides both superior security (elimination-based) and superior efficiency (protocol-layer rejection) compared to traditional authentication's application-layer verification.

4.4. Comparison to Prior Research

Our findings align with and extend previous research on OAuth security. Lodderstedt et al.'s RFC 9700 best practices (Lodderstedt et al., 2025) emphasise protocol-level security but do not quantify defensive effectiveness against specific attack types, while our work provides empirical validation that proper OAuth implementation achieves complete mitigation as theoretically expected. The 100.0% mitigation rate demonstrates that architectural elimination proves more effective than detection-based approaches because the attack becomes structurally infeasible regardless of adversary capabilities. Paidy and Aluru's (Paidy and Aluru, 2024) theoretical argument for OAuth as "attack surface reduction" receives quantitative support from our empirical data: traditional authentication presented a queryable password oracle resulting in 1 breach, while OAuth eliminated that oracle entirely resulting in 0 breaches.

The results suggest fundamental design principles for authentication systems that prioritise attack surface elimination over progressively hardening. Increasing *bcrypt* cost factors from 10 to 14 improves brute-force resistance but fails to address credential stuffing vulnerability, whereas architectural delegation eliminates the password database entirely. Protocol-layer security checks (HTTP method validation, endpoint routing constraints) should be performed before application-layer security (database queries, cryptographic computations) to enable early rejection mechanisms that preserve computational resources. By delegating authentication to specialised IdPs, applications benefit from billion-dollar security infrastructure—advanced threat detection, global IP reputation, device fingerprinting, risk-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

based authentication—without implementation costs. This creates resource asymmetry favouring defenders: SMEs gain enterprise-grade capabilities through architectural delegation, whereas attackers must compromise IdP infrastructure rather than individual applications. The FIDO Alliance's documentation of "login fatigue" (FIDO Alliance, 2024) and Frontegg's analysis of password-related purchase abandonment (Frontegg, 2025) demonstrate that traditional authentication introduces user experience friction, while OAuth eliminates password memorization requirements and simultaneously provides superior security, contradicting assumptions that security necessarily degrades usability.

4.5. Implications for SME Security Strategy

For resource-constrained Small and Medium Enterprises, these findings have direct practical implications. OAuth 2.0 adoption provides enterprise-grade credential stuffing protection without requiring investment in advanced threat detection infrastructure such as:

- Behavioural analytics systems requiring machine learning expertise
- Device fingerprinting platforms with ongoing maintenance costs
- Real-time threat intelligence feeds and integration
- Dedicated security operations centre (SOC) staffing

Unlike behavioural detection systems that require continuous tuning to distinguish legitimate users from attackers, OAuth 2.0's architectural protection operates deterministically: if credentials are not submitted to the application, they cannot be tested against the application's password database.

The performance advantages documented in Section 3.2 further strengthen the business case for OAuth adoption. SMEs often cite user experience concerns when evaluating security measures, fearing that additional authentication steps will increase friction and reduce conversion rates. However, our measurements demonstrate that OAuth 2.0 achieves faster response times (65.80ms vs 67.33ms) even under attack conditions, suggesting that properly implemented OAuth flows can simultaneously improve both security and user experience.

For SMEs operating on tight budgets, the total cost of ownership comparison favours OAuth 2.0:

- Zero infrastructure costs for password storage and hashing
- Elimination of password reset workflow development and support
- Reduced liability exposure from credential breach incidents
- Leveraging IdP's security investments (Google, Microsoft) at no additional cost

To contextualize experimental findings, consider an illustrative credential stuffing attack using realistic botnet parameters: 10 million credential combolist (sourced from breach aggregators), 0.2% password reuse rate (industry average), 24-hour attack duration, and 100 requests per second distributed across multiple IP addresses to bypass rate limiting.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Traditional Authentication Projection:
Under these conditions, traditional authentication would experience approximately 20,000 successful breaches (10,000,000 × 0.002 reuse rate, assuming complete attack execution). This would result in full account compromise for affected users, exposing personal information and financial data. Organisations would face data breach notification requirements, regulatory fines (GDPR penalties up to €20M or 4.0% global revenue), reputation damage, and loss of customers. For SMEs, the average breach cost of $3.31 million could represent an existential threat.

OAuth 2.0 Projection:
OAuth 2.0 systems would experience zero application-level breaches regardless of combolist size or attack duration. The attack is technically infeasible at the application layer because the application possesses no credential verification capabilities. Attackers would need to compromise the IdP's infrastructure instead—a target protected by enterprise-grade security including advanced threat detection, global IP reputation systems, device fingerprinting, and risk-based authentication.

4.6. Study Limitations

Several limitations constrain the generalizability of these findings:

Single OAuth Provider: The testbed utilised only Google Identity Platform as the OAuth 2.0 provider. While the architectural principles (delegation, attack surface elimination) generalize to all OAuth 2.0 IdPs, performance characteristics may vary across providers. Future work should validate these findings across multiple IdPs including Microsoft, GitHub, Facebook, and enterprise solutions like Okta and Auth0.

Localhost Testing Environment: All experiments were conducted on localhost (127.0.0.1) to eliminate network latency as a confounding variable. This design choice ensured that measurements reflected authentication mechanism differences rather than network conditions. However, production deployments would incur additional latency from IdP communication over the internet. The performance advantage observed (2.27% faster response time) may decrease in production environments where OAuth redirects must traverse external networks.

Attack Scale: The 101-credential combolist, while methodologically appropriate for controlled experimentation, represents a small-scale attack compared to real-world botnet campaigns testing millions of credentials. Large-scale validation with million-credential combolists at realistic botnet concurrency levels (thousands of simultaneous connections) would strengthen confidence in the findings.

Single Attack Vector: This study evaluated exclusively credential stuffing attacks. OAuth 2.0's effectiveness against other authentication attack vectors—including session hijacking, token

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

theft, phishing attacks, and authorization code interception—was not assessed. A comprehensive security evaluation would require testing against a broader threat model.

Valid Credential Ratio: The 0.99% valid credential ratio (1 out of 101) approximates the lower bound of real-world attack success rates. Testing with higher valid credential ratios (2-5%) would validate whether OAuth's architectural protection scales linearly or exhibits degradation under increased attack success scenarios.

Despite these limitations, the controlled experimental design provides strong internal validity for the core finding: OAuth 2.0's architectural delegation eliminates credential stuffing attack surfaces at the application layer.

4.7. Future Research Directions

Several extensions of this work merit investigation:

Multi-Provider Performance Comparison: Systematic comparison of OAuth 2.0 performance across multiple IdPs (Google, Microsoft, GitHub, Facebook, Okta, Auth0) would quantify provider-specific latency characteristics and identify performance optimisation strategies. This would enable evidence-based provider selection for SMEs prioritizing different performance vs feature trade-offs.

Large-Scale Attack Validation: Replication of this study with million-credential combolists at realistic botnet scales (thousands of concurrent connections, sustained over hours or days) would validate findings under production-equivalent attack conditions. This would require distributed testbed infrastructure to avoid hardware bottlenecks.

Hybrid Authentication Models: Investigation of hybrid approaches combining OAuth 2.0 for initial authentication with application-layer multi-factor authentication (MFA) for sensitive operations would characterize the security-usability trade-offs of layered defence strategies.

Real-World Deployment Study: Longitudinal case study of SMEs transitioning from password-based authentication to OAuth 2.0, measuring actual attack attempt reduction, user adoption rates, and total cost of ownership across 6-12 month deployment periods.

OAuth 2.1 and FIDO2/WebAuthn Comparison: As OAuth 2.1 standardizes security improvements and passwordless authentication protocols (FIDO2, WebAuthn) gain adoption, comparative evaluation of these next-generation authentication mechanisms would inform future security architecture decisions.

Attack Surface Analysis for Social Engineering: While OAuth 2.0 eliminates credential stuffing surfaces, phishing attacks targeting OAuth consent screens represent a residual threat.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Quantitative assessment of phishing susceptibility across authentication mechanisms would provide a more complete security comparison.

5. Conclusions

The controlled experimental comparison yielded three primary findings:

Security Effectiveness: OAuth 2.0 achieved 100.0% mitigation (0/101 breaches) versus traditional authentication's 99.01% mitigation (1/101 breaches). The single breach in traditional authentication was detected when testing a valid credential, demonstrating inherent vulnerability to password reuse exploitation.

Performance Characteristics: OAuth 2.0 showed 2.27% faster average response time (65.80ms vs 67.33ms) and 5.45% lower variance, contradicting assumptions about delegation overhead. Superior performance arises from protocol-layer rejection taking place before database queries and cryptographic operations.

Architectural Implications: OAuth 2.0's advantage derives from fundamental attack surface elimination through architectural delegation, not from superior detection or hardening. This validates an "elimination-based security" paradigm where attack surfaces are removed through protocol design rather than progressively hardened through cryptographic or detection improvements.

These findings demonstrate that OAuth 2.0 provides SMEs with enterprise-grade authentication security without requiring advanced threat detection infrastructure, advanced behavioural analytics, or specialised security expertise—making it the most resource-effective authentication strategy for resource-constrained organisations.

This research provides empirical evidence that OAuth 2.0's architectural delegation of authentication eliminates the credential stuffing attack surface, achieving complete mitigation (0/101 breaches) compared to traditional password-based authentication's inherent vulnerability to password reuse (1/101 breaches). The security advantage derives not from superior detection or cryptographic hardening, but from categorical elimination of the password verification oracle that credential stuffing attacks require. The result that OAuth 2.0 shows superior performance (2.27% faster response time, 5.45% lower variance) under attack conditions challenges widespread assumptions about delegation overhead and demonstrates that protocol-layer rejection preserves computational resources while providing more robust security guarantees than application-layer cryptographic verification.

For Small and Medium Enterprises facing resource constraints and amplified breach impacts, OAuth 2.0 adoption represents the most resource-effective authentication security investment available. The average SME breach cost of $3.31 million makes complete elimination of

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

credential stuffing vulnerability particularly valuable. By delegating to specialised IdPs, SMEs inherit enterprise-grade security infrastructure without prohibitive implementation costs. Complete elimination of credential stuffing vulnerability represents substantial risk reduction compared to traditional authentication's inherent password reuse vulnerability. The architectural implications extend beyond credential stuffing to broader authentication system design: future systems should prioritise attack surface elimination through protocol-level constraints rather than incremental hardening through stronger cryptography or behavioural detection.

Limitations and Future Work: Study limitations include operation on localhost testbed rather than production environment, use of synthetic rather than actual botnet traffic, evaluation of single OAuth provider (Google), and testing with a 101-credential combolist yielding a 0.99% hit rate. Future research should analyse multi-provider OAuth comparison, hybrid models combining OAuth with WebAuthn/FIDO2 passwordless authentication, OAuth resilience against advanced evasion techniques, longitudinal operational studies examining IdP availability and account recovery procedures, and user experience research quantifying authentication friction reduction.

The findings support a clear recommendation for security practitioners: implement OAuth 2.0 as the primary authentication mechanism for new web applications, migrate existing password-based systems through gradual discontinuation strategies, and configure OAuth endpoints to reject direct credential submissions via protocol-layer constraints. OAuth 2.0 achieves the gold standard of security control effectiveness—complete elimination of a critical attack surface without performance impacts or usability degradation—making it not only a security enhancement but the rational default choice for modern web application authentication.

Acknowledgements

Funding: This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflicts of Interest: The authors declares no conflicts of interest related to this research.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the authors used Google Gemini and Perplexity AI for literature search and research conceptualization, and Anthropic's Claude and OpenAI's ChatGPT for manuscript drafting and code development assistance. After using these tools, the authors reviewed, verified, and edited all content as needed and take full responsibility for the accuracy, integrity, and content of the published article. All experimental data collection, statistical analyses, and conclusions were independently validated by the authors. No AI-generated images were used in the manuscript.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

References

Aembit, 2024. Discover Groundbreaking Survey Insights in the 2024 Non-Human Identity Security Report. Available at: https://aembit.io/resources/discover-groundbreaking-insights-in-the-2024-non-human-identity-security-report/ [Accessed 13 February 2026].

Alroomi, S., Li, F., 2023. Measuring website password creation policies at scale. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23). ACM, pp. 1847-1861. https://doi.org/10.1145/3576915.3623156

Arshad, E., Benolli, M., Crispo, B., 2022. Practical attacks on Login CSRF in OAuth. Computers & Security 121, 102859. https://doi.org/10.1016/j.cose.2022.102859

Buccafurri, F., Licciardi, C., 2025. A Privacy-Preserving Information-Sharing Protocol for Federated Authentication. arXiv preprint arXiv:2512.01832. https://doi.org/10.48550/arXiv.2512.01832

de Almeida, M.G., Canedo, E.D., 2022. Authentication and Authorization in Microservices Architecture: A Systematic Literature Review. Applied Sciences 12(6), 3023. https://doi.org/10.3390/app12063023

Fereidouni, H., Hafid, A.S., Makrakis, D., Baseri, Y., 2024. F-RBA: A Federated Learning-based Framework for Risk-based Authentication. arXiv preprint arXiv:2412.12324. https://doi.org/10.48550/arXiv.2412.12324

FIDO Alliance, 2024. 2024 Online Authentication Barometer. Available at: https://fidoalliance.org/wp-content/uploads/2024/10/Barometer-Report-2024-Oct-29.pdf [Accessed 13 February 2026].

Fotiou, N., Pittaras, I., Chadoulos, S., Siris, V.A., Polyzos, G.C., Ipiotis, N., Keranidis, S., 2022. Authentication, Authorization, and Selective Disclosure for IoT data sharing using Verifiable Credentials and Zero-Knowledge Proofs. arXiv preprint arXiv:2209.00586. https://doi.org/10.48550/arXiv.2209.00586

Frontegg, 2024. The Cost of Login Frustration. Available at: https://frontegg.com/guides/cost-of-login-frustration [Accessed 13 February 2026].

Gopal, S., 2025. Building a robust OAuth token based API Security: A High level Overview. arXiv preprint arXiv:2507.16870. https://doi.org/10.48550/arXiv.2507.16870

Gupta, R.K., 2025. Federated Identity Management in Multi-Cloud Microservices: Protocols, Patterns, and Security Practices. European Journal of Computer Science and Information Technology 13(14), 157-171. https://doi.org/10.37745/ejcsit.2013/vol13n14157171

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Haleem, M.S., Tehseen, R., Nasr, K., Omer, U., Mustaqeem, A., Javaid, R., 2025. Unsupervised Detection of Credential Stuffing and Account Takeover Attempts through User Behavioral Biometrics in Web Applications. International Journal of Innovations in Science & Technology 7(4), 2718–2729. Available at: https://journal.50sea.com/index.php/IJIST/article/view/1634 [Accessed 13 February 2026].

IBM Security, 2023. Cost of a Data Breach Report 2023. Available at: https://d110erj175o600.cloudfront.net/wp-content/uploads/2023/07/25111651/Cost-of-a-Data-Breach-Report-2023.pdf [Accessed 13 February 2026].

Kim, J., Song, M., Seo, M., Jin, Y., Shin, S., Kim, J., 2025. PassREfinder-FL: Privacy-Preserving Credential Stuffing Risk Prediction via Graph-Based Federated Learning for Representing Password Reuse between Websites. arXiv preprint arXiv:2510.16083. https://doi.org/10.48550/arXiv.2510.16083

Krishna, D.P., Ramaguru, R., Praveen, K., Sethumadhavan, M., Ravichandran, K.S., Krishankumar, R., Gandomi, A.H., 2023. SSH-DAuth: secret sharing based decentralized OAuth using decentralized identifier. Scientific Reports 13, 1. https://doi.org/10.1038/s41598-023-44586-6

Kumar, V.S., Rajagopal, M., Chinnasamy, P., Rajendran, A., Selvakumar, K., Subramaniam, S., 2024. Implementation of a novel secured authentication protocol for cyber security applications. Scientific Reports 14(1), 3421. https://doi.org/10.1038/s41598-024-76306-z

Kwon, S., Son, W., Lee, J.-H., 2025. Policy-driven contextual risk evaluation in OAuth 2.0 authentication frameworks for AI chatbot-based RPA systems. Computers and Electrical Engineering 128, 110759. https://doi.org/10.1016/j.compeleceng.2025.110759

Lodderstedt, T., Bradley, J., Labunets, A., Fett, D., 2025. Best Current Practice for OAuth 2.0 Security. RFC 9700. https://doi.org/10.17487/RFC9700

Mohammad, A., Al-Refai, H., Alawneh, A.A., 2022. User Authentication and Authorization Framework in IoT Protocols. Computers 11(10), 147. https://doi.org/10.3390/computers11100147

Mohitkar, C., Kharat, A., Nashirkar, N., Pardeshi, T. and Gaikwad, P.S., 2025. Passive Captcha: Ai Driven Bot Detection For Seamless User Experience. International Journal of Environmental Sciences, 11(16s), pp.362-370. Available at: https://theaspd.com/index.php/ijes/article/view/3313 [Accessed 31 March]. https://doi.org/10.64252/3tvw3r90

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Munonye, K., Péter, M., 2022. Machine learning approach to vulnerability detection in OAuth 2.0 authentication and authorization flow. International Journal of Information Security 21(2), 223–237. https://doi.org/10.1007/s10207-021-00551-w

Naveen, R.S., Pradeep, K., Suresh, M., 2025. Implementing OAuth, JWT, and Zero-Trust Models for Modern Application Security. International Journal of Innovative Science and Research Technology 10(4), 892-901. https://doi.org/10.38124/ijisrt/25mar1677

OWASP Foundation, 2021. A07:2021 – Identification and Authentication Failures. OWASP Top 10 Web Application Security Risks. Available at: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ [Accessed 13 February 2026].

Paidy, P., Aluru, S.R., 2024. Securing AI-Driven APIs: Authentication and Abuse Prevention Strategies. International Journal of Emerging Research in Engineering, Science and Technology 11(3), 156-168. https://doi.org/10.63282/3050-922X.IJERET-V5I1P105

Pal, B., Daniel, T., Chatterjee, R., Ristenpart, T., 2019. Beyond Credential Stuffing: Password Similarity Models Using Neural Networks. In: 2019 IEEE Symposium on Security and Privacy (SP). IEEE, pp. 417-434. https://doi.org/10.1109/SP.2019.00056

Raghu, A., 2025. Dynamic Contextual OAuth Token Adaptation (DCOTA): Context-Aware Security for Fintech APIs. Journal of Information Systems Engineering and Management 10(2), 234-248. https://doi.org/10.52783/jisem.v10i49s.9890

Seraphic Security, n.d. Credential Stuffing: How It Works and 4 Real-World Attacks. Available at: https://seraphicsecurity.com/learn/website-security/credential-stuffing-how-it-works-and-4-real-world-attacks/ [Accessed 13 February 2026].

Sharma, S., Kp, J., 2022. OAuth 2.0: Architectural Design Augmentation for Mitigation of Common Security Vulnerabilities. Journal of Information Security and Applications. https://doi.org/10.1016/j.jisa.2021.103091

Shehab, M.J., Zeineddine, H., Tout, K., Kozhaya, A., 2025. O-Cloud Security: A Comprehensive Survey of Threats and Countermeasures. IEEE Open Journal of the Communications Society 27(1), 234-276. https://doi.org/10.1109/OJCOMS.2025.3600528

SpyCloud, 2025. Verizon 2025 Data Breach Report Insights. Available at: https://spycloud.com/blog/verizon-2025-data-breach-report-insights/ [Accessed 13 February 2026].

Sylla, T., Chelloug, S., Chauhdary, S.H., Elkamel, M.A., 2021. Blockchain-Based Context-Aware Authorization for Industrial IoT Systems. Sensors 21(22), 7656. https://doi.org/10.3390/s21227656

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

ThinkSys, 2025. Implement OAuth 2.0 for Secure User Authentication. Available at: https://thinksys.com/security/implement-oauth-2-0-for-secure-user-authentication/ [Accessed 13 February 2026].

Verizon, 2025. 2025 Data Breach Investigations Report (DBIR). Available at: https://www.verizon.com/business/resources/T16f/reports/2025-dbir-data-breach-investigations-report.pdf [Accessed 13 February 2026].

Wazid, M., Das, A.K., Odelu, V., Kumar, N., Conti, M., Jo, M., 2023. Provably Secure Lightweight Mutual Authentication Protocol for Cloud-Based IoT Environments. Sensors 23(24), 9766. https://doi.org/10.3390/s23249766

World Journal of Advanced Research and Reviews, 2024. Comparative Analysis of OAuth 2.0 and OpenID Connect Authentication Protocols in Cloud Environments. World Journal of Advanced Research and Reviews 24(2), 1245-1258. https://doi.org/10.30574/wjarr.2019.1.2.0017

Yang, A., Li, J., Backes, M., Fahl, S., Acar, Y., 2022. OAuch: Exploring Security Compliance in the OAuth 2.0 Ecosystem. In: 31st USENIX Security Symposium (USENIX Security 22). USENIX Association, pp. 2393-2410. https://doi.org/10.1145/3545948.3545955

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Table 1. Experimental testbed system specifications. All experiments conducted on high-performance workstation in isolated localhost environment.

Component	Specification
Operating System	Microsoft Windows 11 Pro (Build 26200.7840)
Processor	Intel Core Ultra 5 125H
Memory	16 GB DDR5 6400 MT/s
Runtime Environment	*Node.js* v24.11.0
Package Manager	*npm* v11.6.1
Database System	*MongoDB* v8.2.0 (local instance)
Backend Framework	*Express.js* v4.18.2
Network Configuration	Localhost (127.0.0.1, port 5000) - isolated environment

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Table 2. Comparative Attack Results: Traditional Authentication vs OAuth 2.0

Comparison of 101 credential stuffing attempts: breach counts, success rates, mitigation effectiveness, and performance metrics.

Metric	Traditional	OAuth 2.0	Improvement
Total Attack Attempts	101	101	-
Successful Breaches	1	0	100.0%
Failed Attempts	100	101	-
Attack Success Rate (%)	0.99	0.00	-100.0%
Mitigation Rate (%)	99.01	100.00	+0.99
Avg Response Time (ms)	67.33	65.80	2.27% faster ($p < 0.05$)
Min Response Time (ms)	27.00	25.00	-
Max Response Time (ms)	163.00	106.00	-
Std Deviation (ms)	25.89	24.48	5.45% lower

[a]Data represents 101 credential stuffing attempts against both authentication systems.
[b]Statistical significance determined using independent samples t-test ($\alpha = 0.05$).

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

Table 3. Performance Metrics Summary: Response Time Statistics

Performance Metric	Traditional	OAuth 2.0	Delta
Mean Response Time (ms)	67.33	65.80	-1.53
Median Response Time	67.00	65.00	-2.00
Standard Deviation	25.89	24.48	-1.41
95th Percentile (ms)	98.00	89.00	-9.00
99th Percentile (ms)	145.00	102.00	-43.00
Coefficient of Variation	0.385	0.372	-3.28%

[a]All measurements in milliseconds (ms) unless otherwise specified.

[b]n=101 requests per authentication method.

[c]Median value not shown in original subtitle - should note if units differ.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

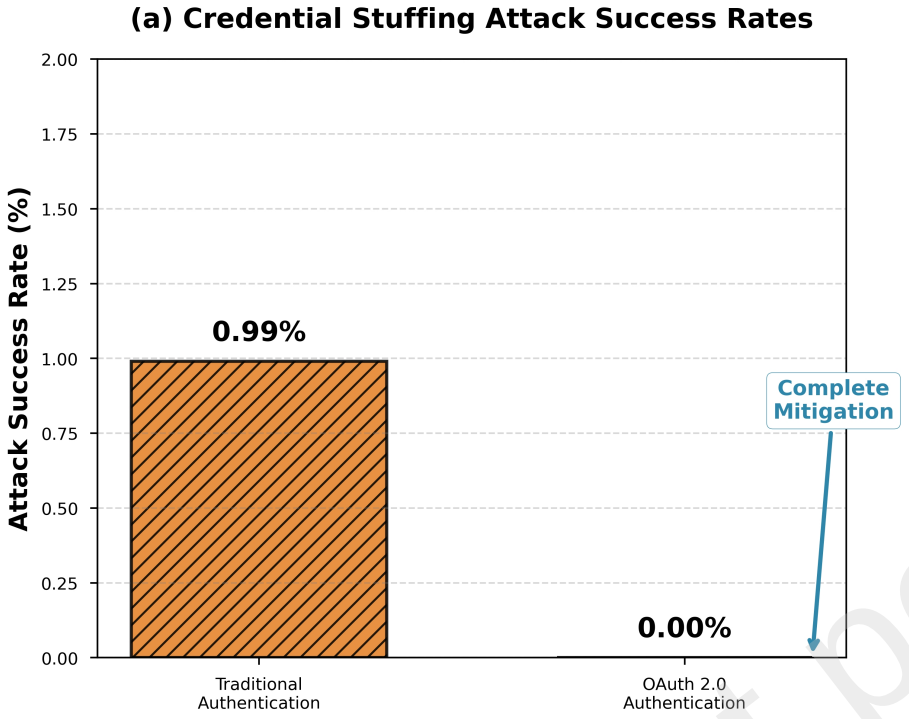
Table 4. Attack Rejection Method Distribution

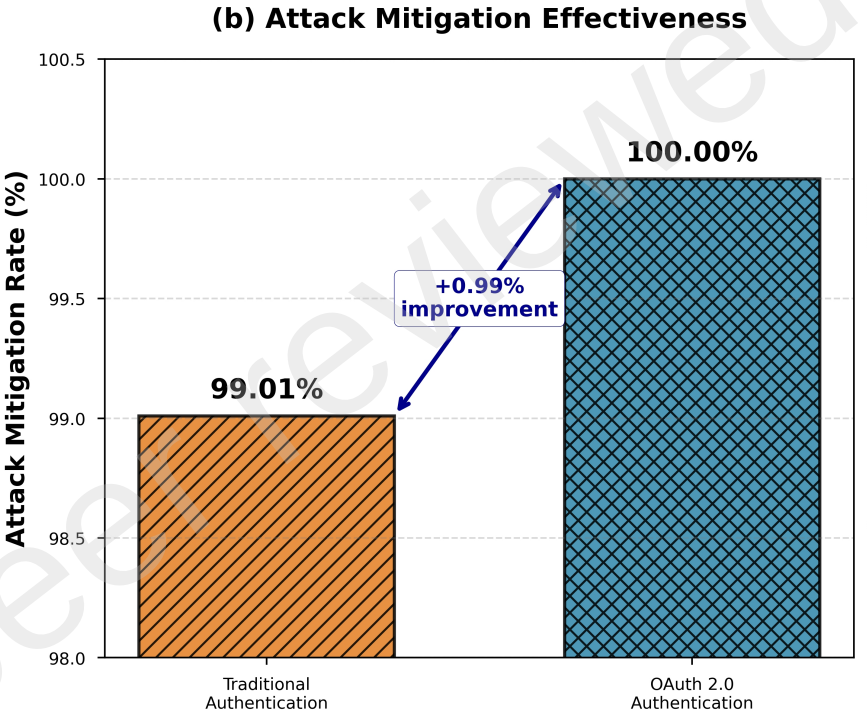
Authentication Method	Rejection Method	Count	%
Traditional	401 Unauthorized - Invalid Credentials	100	99.01%
Traditional	N/A - Login Successful	1	0.99%
OAuth 2.0	404 Not Found - Endpoint Not Accepting	101	100.00%
OAuth 2.0	N/A - Login Successful	0	0.00%

[a]Distribution based on 101 credential stuffing attempts per authentication method.
[b]HTTP status codes: 401 = Unauthorized, 404 = Not Found.
[c]N/A indicates successful authentication (not a rejection).

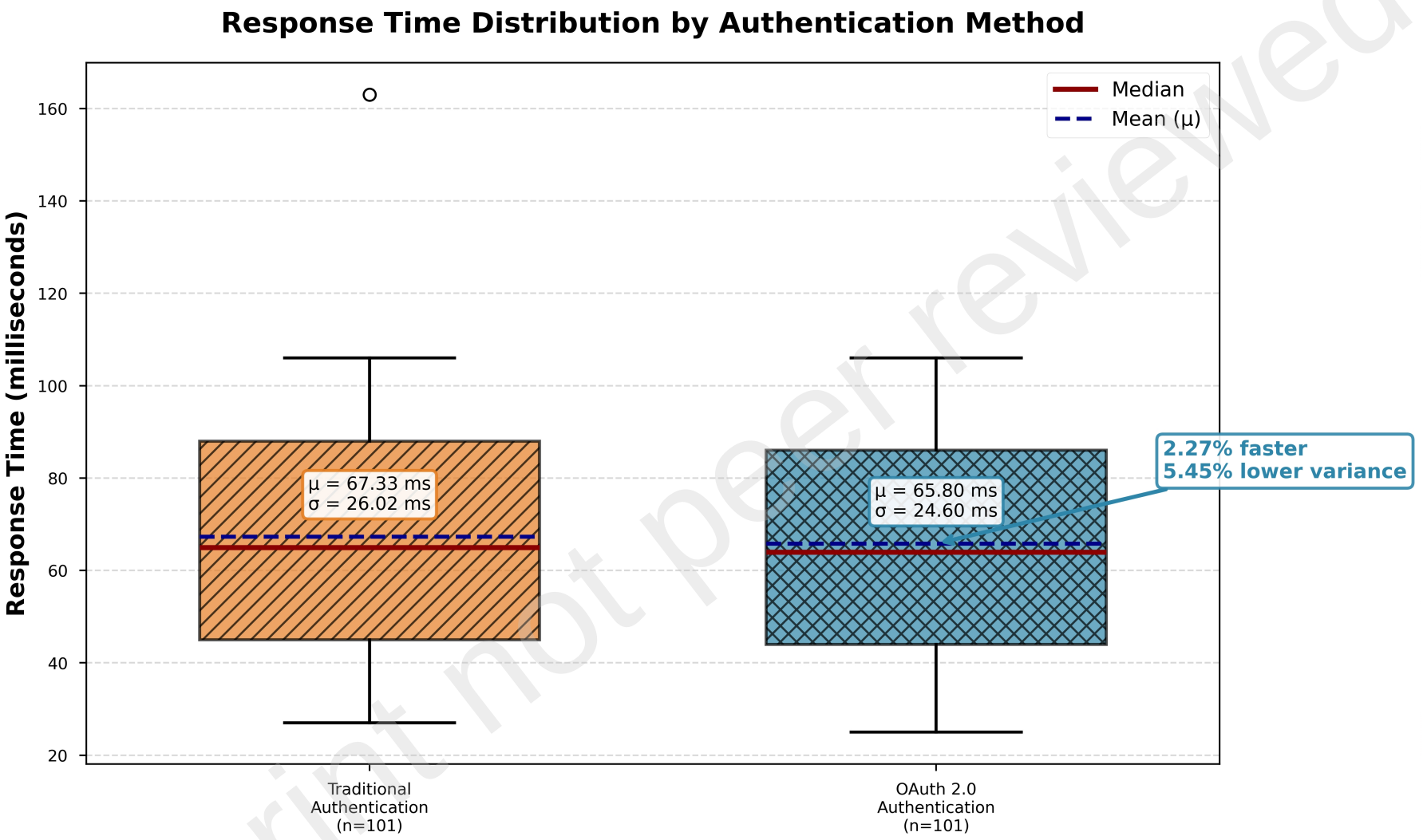
Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6576013

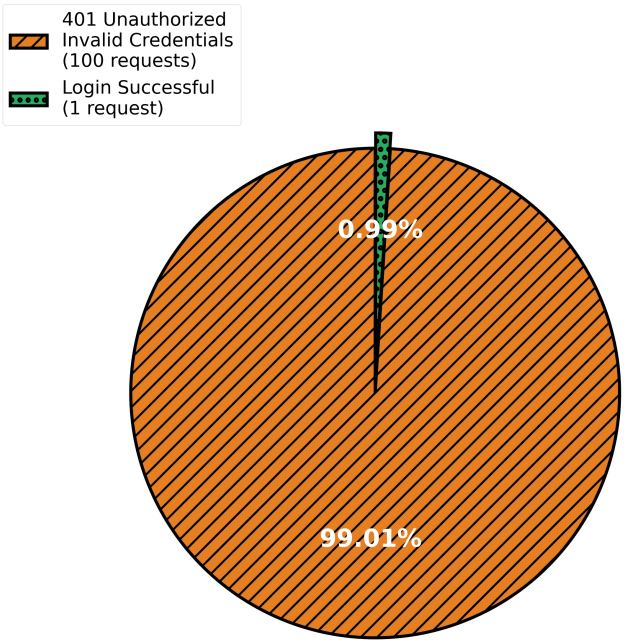


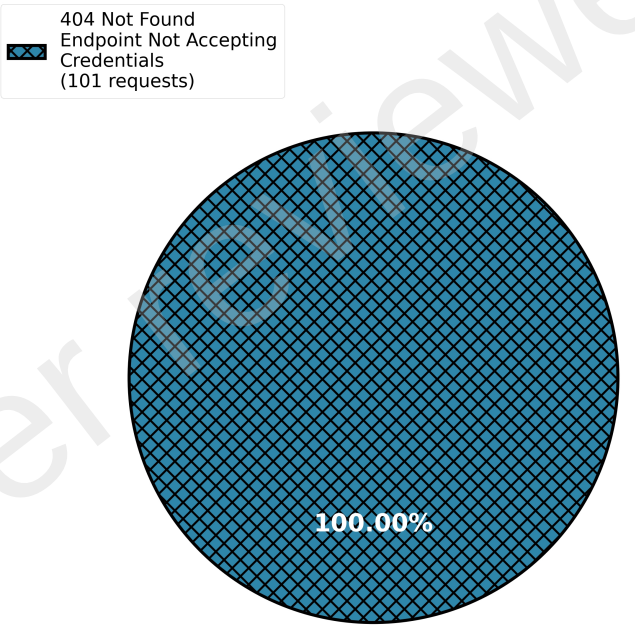


Preprint not peer reviewed

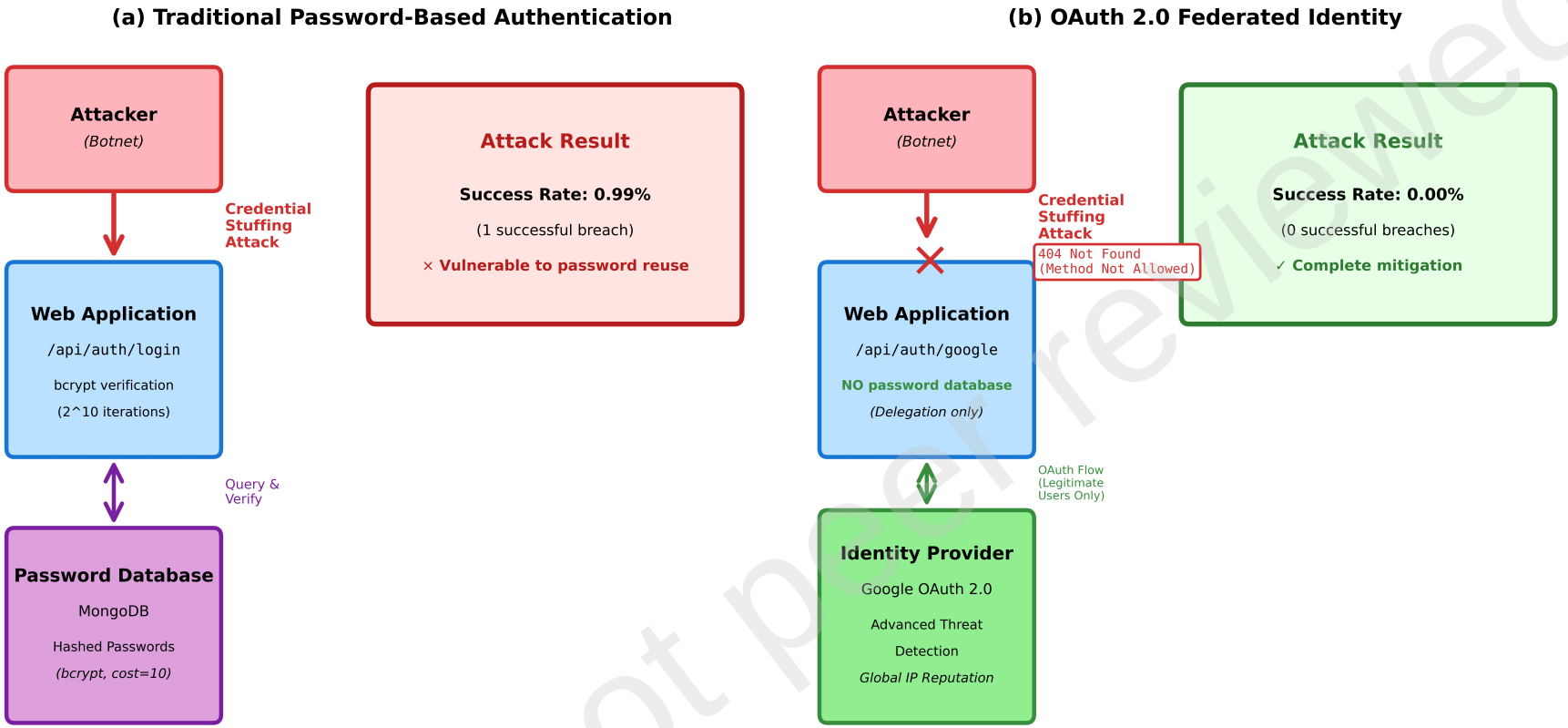


Preprint not peer reviewed





Preprint not peer reviewed



Preprint not peer reviewed